

# PropNest

## Introduction:

PropNest is a modern Full-Stack Real Estate Property Management Platform developed to simplify and digitalize the property buying, selling, and management process by seamlessly connecting buyers and agents through a secure and interactive online ecosystem. Built using the MERN stack (MongoDB, Express.js, React.js, and Node.js), the platform integrates advanced technologies such as Socket.io for realtime messaging, Leaflet for location-based property mapping, React Query for efficient state management, and Tailwind CSS for a responsive user interface. PropNest enables buyers to search and filter properties based on location, price, and amenities, compare listings, save favorites, schedule site visits, and communicate directly with agents, while agents benefit from dedicated dashboards for property management, lead tracking, analytics, and performance monitoring. By combining role-based authentication, dynamic data flow, and scalable architecture, PropNest addresses common challenges in traditional real estate systems such as fragmented communication, inefficient lead handling, and lack of transparency, ultimately providing a smarter, faster, and more user-friendly real estate experience.

## SCENARIO Based Case Study:

In a typical real estate scenario, a buyer searching for a suitable property often faces multiple challenges such as limited property visibility, dependence on brokers, delayed communication, and difficulty comparing various options. For instance, a working professional looking for a 2BHK apartment within a fixed budget in a preferred location may need to visit several agents, gather fragmented information, and manually evaluate different properties. This traditional process is often timeconsuming, less transparent, and inefficient.

With PropNest, the buyer can access a centralized digital platform where they can search properties using advanced filters such as location, price range, property category, and amenities. The interactive map view helps the buyer explore properties geographically, while features like saved listings, property comparison, and site visit scheduling simplify decision-making. Once a suitable property is identified, the buyer can instantly communicate with the respective agent through real-time messaging, eliminating delays and improving engagement.

From the agent's perspective, PropNest provides a powerful dashboard to manage property listings, track buyer inquiries, monitor site visit requests, and analyze performance metrics. For example, when the buyer sends an inquiry, the agent receives it immediately, responds efficiently, and tracks the lead throughout the sales process. This scenario highlights how PropNest transforms traditional property

transactions into a seamless, transparent, and technology-driven ecosystem that benefits both buyers and agents through speed, accessibility, and smarter real estate management.

## SYSTEM REQUIREMENTS:

### Software Requirements:

To ensure smooth development, deployment, and usage of the PropNest Real Estate Property Management Platform, specific system prerequisites must be fulfilled. These requirements are divided into software and hardware specifications to support the platform's full-stack MERN architecture, real-time communication, interactive mapping, and responsive user experience.

#### 1. Software Requirements

These are the essential tools, frameworks, and platforms required to efficiently develop, test, and run the PropNest application.

- **Operating System:** Windows 10/11, macOS, or Linux — Supports crossplatform development, deployment, and testing.
- **Node.js (v16 or above):** Provides the runtime environment for both frontend and backend development, including React application execution and Express server operations.
- **npm (v8 or above):** Package manager used to install and manage dependencies for React, Node.js, Socket.io, Tailwind CSS, and other libraries.
- **React.js (React 18 with Vite):** Frontend framework used to build dynamic, interactive, and responsive user interfaces.
- **Express.js:** Backend web framework for creating RESTful APIs, managing routes, and handling server-side logic.
- **MongoDB:** NoSQL database used to store user profiles, property listings, agent details, leads, messages, and analytics data.
- **Mongoose:** ODM library for MongoDB schema design and database interaction.
- **Socket.io:** Enables real-time communication between buyers and agents for live messaging and notifications.
- **Leaflet / React Leaflet:** Used for geolocation-based property mapping and interactive map visualization.
- **Tailwind CSS & Shadcn UI:** Provides responsive styling, UI components, and enhanced user experience.
- **Browser:** Google Chrome / Mozilla Firefox / Microsoft Edge (latest versions) — For real-time UI rendering and testing.

- **Postman:** API testing tool for validating backend routes such as authentication, properties, leads, and messaging.
- **Visual Studio Code:** Recommended code editor with Git integration, terminal support, and extension ecosystem for MERN development.
- **Git & GitHub:** Version control and collaborative project management.

## 2. Hardware Requirements

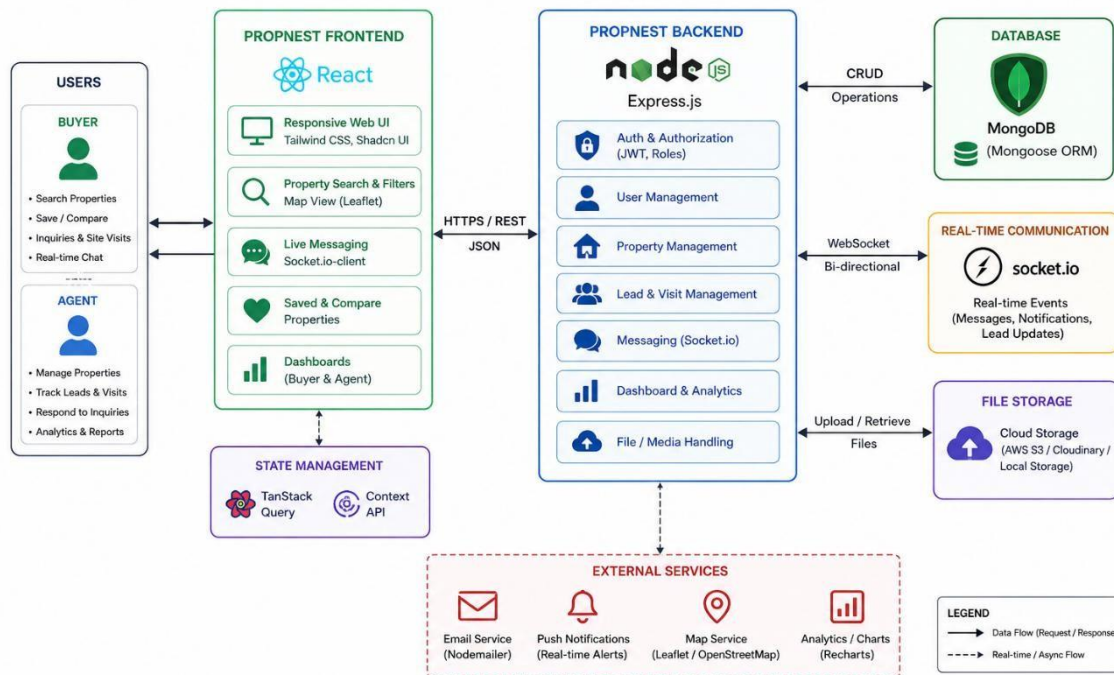
These specifications define the minimum and recommended hardware setup for effective development and deployment of PropNest.

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or better — Ensures efficient full-stack development, server execution, and browser multitasking.
- **RAM:** Minimum 8 GB (16 GB recommended) — Supports simultaneous execution of frontend, backend, MongoDB, browser testing, and development tools.
- **Storage:** Minimum 2 GB free disk space (SSD recommended) — Required for project files, package installations, MongoDB storage, and development environment setup.
- **Display:** 1366x768 or higher (Full HD recommended) — Provides better workspace for coding, dashboard management, and UI visualization.
- **Internet Connection:** Stable broadband connection — Necessary for package downloads, cloud database integration (MongoDB Atlas), deployment, and version control.

## PROJECT ARCHITECTURE:

## TECHNICAL ARCHITECTURE:

## PropNest – Technical Architecture



In the PropNest architecture, the workflow begins with the user, who can either be a Buyer or an Agent, interacting with the PropNest Frontend through a modern web interface developed using React.js, Tailwind CSS, Shadcn UI, and Vite. Buyers use the platform to search for properties, apply filters, view maps, compare listings, save properties, schedule site visits, and communicate with agents, while agents use dedicated dashboards to manage property listings, track leads, and monitor analytics.

The PropNest Frontend sends user requests to the PropNest Backend, developed using Node.js and Express.js, through secure RESTful APIs. The backend handles authentication, property management, buyer-agent interactions, lead processing, dashboard analytics, and real-time communication. JWT-based role authentication ensures secure access control for both buyers and agents, while middleware manages protected routes and user authorization.

The backend communicates with MongoDB, which serves as the primary database for storing structured platform data such as user profiles, property details, leads, saved properties, chat history, and dashboard statistics. Mongoose is used to manage schema design and streamline data operations between the application and database.

For real-time messaging and instant notifications, Socket.io acts as a communication layer between buyers and agents, enabling live chat, inquiry updates, and lead notifications without requiring page refreshes. This ensures seamless interaction and enhances user engagement.

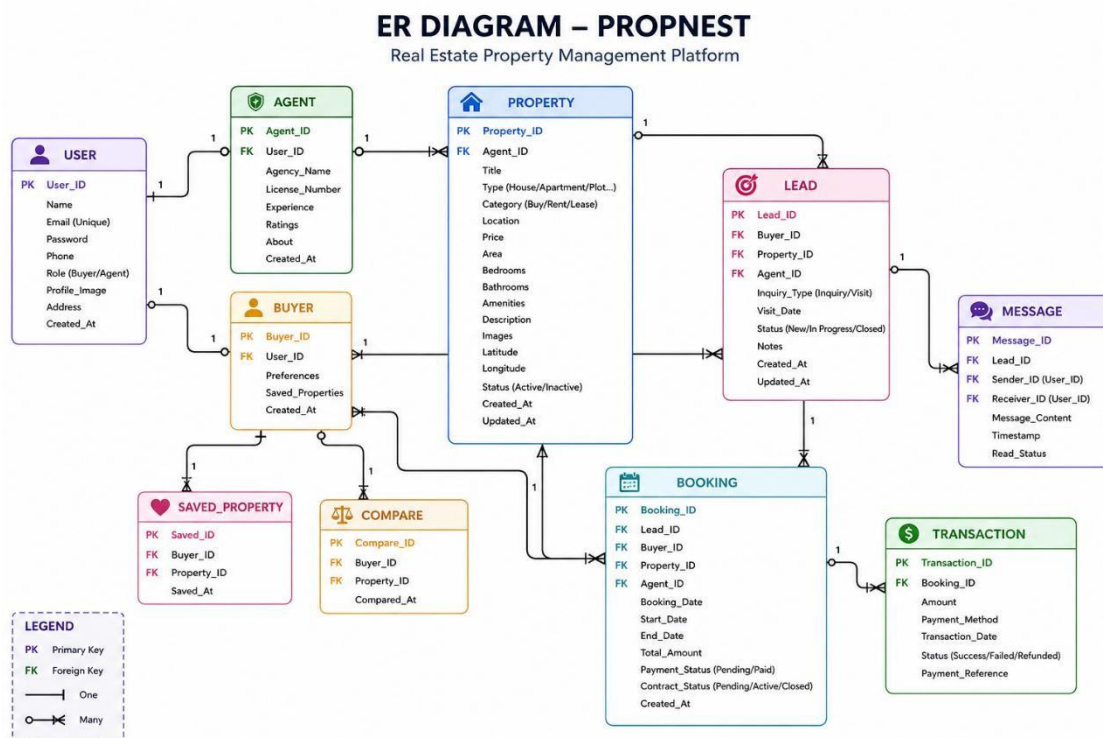
Additionally, file storage systems or cloud media services can be integrated for storing uploaded property images, agent profile photos, and property-related documents.

These files are linked to property records and displayed dynamically on the frontend for better visual representation.

Once the backend processes requests such as property searches, login authentication, lead creation, or messaging, it sends structured responses back to the frontend. The frontend then dynamically updates the user interface, displaying relevant property data, dashboards, chat messages, or analytics in real time.

Overall, the PropNest technical architecture follows a scalable full-stack MERN ecosystem where the Frontend handles user interaction, the Backend manages business logic and security, MongoDB stores core data, Socket.io powers real-time communication, and optional file storage supports media management—creating a seamless, interactive, and efficient real estate management platform.

## ER DIAGRAM:



The Entity-Relationship (ER) diagram for PropNest represents the major entities and their relationships within the real estate property management system. It visually defines how buyers, agents, properties, leads, bookings, and communication modules interact to create a seamless digital property ecosystem.

### 1. Entities:

- **User:**
- Represents all registered platform users, including Buyers and Agents, with role-based differentiation.

**Attributes:** User\_ID (PK), Name, Email, Password, Phone, Role (Buyer/Agent), Profile\_Image, Address.

- **Agent:**
  - Represents specialized real estate agents who manage properties and interact with buyers.
  - Attributes:** Agent\_ID (PK), User\_ID (FK), Agency\_Name, License\_Number, Experience, Ratings.
- **Buyer:**
  - Represents users searching for or purchasing properties.
  - Attributes:** Buyer\_ID (PK), User\_ID (FK), Preferences, Saved\_Properties.
- **Property:**
  - Represents property listings uploaded by agents.
  - Attributes:** Property\_ID (PK), Agent\_ID (FK), Title, Type, Category, Location, Price, Area, Amenities, Images, Status.
- **Lead:**
  - Represents buyer inquiries, site visit requests, or interest in a specific property.
  - Attributes:** Lead\_ID (PK), Buyer\_ID (FK), Property\_ID (FK), Agent\_ID (FK), Inquiry\_Type, Visit\_Date, Status, Notes.
- **Message:**
  - Represents real-time chat communication between buyers and agents.
  - Attributes:** Message\_ID (PK), Lead\_ID (FK), Sender\_ID (FK), Receiver\_ID (FK), Message\_Content, Timestamp, Read\_Status.
- **Booking:**
  - Represents finalized property booking or lease agreements.
  - Attributes:** Booking\_ID (PK), Buyer\_ID (FK), Property\_ID (FK), Agent\_ID (FK), Booking\_Date, Payment\_Status, Contract\_Status.
- **Transaction:**
  - Represents payment and financial records related to bookings.
  - Attributes:** Transaction\_ID (PK), Booking\_ID (FK), Amount, Payment\_Method, Transaction\_Date, Status.

## 2. Relationships:

- **Agent – Property:**
  - One Agent can manage multiple Properties, while each Property belongs to one Agent.
- **Buyer – Lead:**
  - One Buyer can generate multiple Leads, while each Lead is linked to one Buyer.
- **Property – Lead:**
  - One Property can receive multiple buyer inquiries or visit requests.
- **Buyer – Property (Saved/Compare):**

- Buyers can save or compare multiple properties through many-to-many relationships.
- **Lead – Message:**
- Each Lead can have multiple Messages exchanged between Buyer and Agent.
- **Lead – Booking:**
- A qualified Lead can be converted into one Booking.
- **Booking – Transaction:**
- Each Booking can have one or multiple financial Transactions.
- **Supplier Equivalent (Optional Media/Service Providers):**
- External services such as cloud storage, payment gateways, or map APIs may support system operations.

### 3. Attributes:

Each entity contains unique attributes that define its role within the system. For example, the Property entity includes pricing, amenities, category, and geolocation, while the Lead entity tracks inquiry progress and visit scheduling.

### 4. Primary Keys:

Each entity contains a unique primary key for identification:

- User\_ID
- Agent\_ID
- Buyer\_ID
- Property\_ID
- Lead\_ID
- Message\_ID
- Booking\_ID
- Transaction\_ID

### 5. Foreign Keys:

Foreign keys establish relational integrity between connected entities:

- User\_ID in Agent/Buyer
- Agent\_ID in Property
- Buyer\_ID, Property\_ID, Agent\_ID in Lead
- Lead\_ID in Message
- Booking\_ID in Transaction

## **Key Features:**

The key features of PropNest are designed to provide a seamless, secure, and technology-driven real estate experience for both buyers and agents. As a full-stack property management platform, PropNest combines intelligent property discovery, real-time communication, and advanced management tools to simplify the buying, selling, and property handling process.

### **User Registration and Role-Based Profiles:**

PropNest allows users to create secure accounts as either Buyers or Agents through JWT-based authentication. Buyers can manage saved properties, preferences, and inquiries, while Agents can maintain professional profiles, agency details, and property portfolios.

### **Advanced Property Search and Filtering:**

Users can search, browse, and filter properties based on location, price range, property type, category, amenities, and geolocation. Interactive map integration using Leaflet enhances property discovery with location-based visualization.

### **Property Management System:**

Agents can create, update, and manage property listings with complete CRUD functionality, including property descriptions, pricing, images, amenities, and status updates.

### **Real-Time Messaging and Communication:**

PropNest enables instant buyer-agent communication through Socket.io-powered live chat, allowing users to inquire about properties, schedule site visits, and receive real-time responses.

### **Saved Properties and Comparison:**

Buyers can save favorite properties for future reference and compare multiple listings side-by-side to make informed purchasing decisions.

### **Lead Tracking and Site Visit Requests:**

The platform allows buyers to submit inquiries or request property visits, while agents can monitor and manage leads, visit schedules, and inquiry statuses through dedicated dashboards.

### **Dashboard Analytics:**

Agents gain access to advanced dashboards featuring property performance metrics, lead conversion tracking, revenue insights, booking trends, and top-performing listings using Recharts.



### **Booking and Transaction Readiness:**

While currently focused on lead generation, PropNest is structured to support future integration of formal booking systems, payment gateways, and transaction management.

### **Responsive and User-Friendly Interface:**

Built with React, Tailwind CSS, and Shadcn UI, PropNest offers a modern, mobileresponsive, and accessible interface for seamless use across devices.

### **Security and Data Management:**

The platform ensures secure user authentication, protected routes, encrypted credentials, and scalable MongoDB data storage for user profiles, properties, leads, and communications.

## **ROLES AND RESPONSIBILITIES:**

### **User:**

#### **Registration and Account Management:**

Buyers are responsible for creating and maintaining their PropNest accounts by providing accurate personal details, setting preferences, managing profile information, and ensuring password security. They can update account details, save favorite properties, and manage their activity history. **Property Search and**

#### **Exploration:**

Buyers can search, browse, and filter available properties based on criteria such as location, budget, property type, amenities, and category. They are responsible for selecting suitable properties, comparing listings, saving preferences, and exploring property details through interactive maps and dashboards. **Lead Generation**

#### **and Site Visit Requests:**

Buyers are responsible for initiating inquiries, requesting property visits, and communicating their requirements clearly to agents. They can schedule visits, track inquiry progress, and manage their property interest efficiently through the platform.

#### **Real-Time Communication:**

Buyers use PropNest's messaging system to communicate directly with agents regarding property details, negotiations, site visits, or clarifications. They should use these communication channels responsibly and follow platform guidelines.

#### **Booking and Transaction Participation:**

In future-enabled modules, buyers will be responsible for confirming bookings, completing payment procedures, and managing property transaction workflows securely.

#### **Feedback and Reviews:**

Buyers can provide honest feedback, ratings, and reviews regarding agents, properties, or overall platform experiences to support transparency and help future users make informed decisions.

## **Agent:**

### **Registration and Professional Profile Management:**

Agents are responsible for registering as authorized property managers, maintaining accurate professional details such as agency information, certifications, and property portfolios, and ensuring profile authenticity. **Property Listing and**

### **Management:**

Agents manage the complete lifecycle of property listings, including adding, updating, deleting, and maintaining accurate information such as pricing, amenities, location, images, and availability status.

### **Lead Tracking and Buyer Interaction:**

Agents are responsible for monitoring buyer inquiries, responding promptly to site visit requests, managing lead statuses, and maintaining professional communication to improve conversion rates. **Performance Monitoring:**

Agents utilize dashboard analytics to track property performance, buyer engagement, lead generation trends, and personalized business insights for strategic decisionmaking.

### **Booking and Deal Finalization:**

With future system enhancements, agents will be responsible for approving bookings, processing property sales or rentals, and managing transaction-related workflows.

## **Admin:**

### **System Management:**

The admin oversees the complete PropNest platform by managing system configurations, monitoring platform performance, ensuring uptime, maintaining scalability, and supervising feature deployment.

### **User and Agent Management:**

Admins manage buyer and agent registrations, account verification, role authorization, moderation, approvals, suspensions, and policy enforcement to maintain platform integrity.

### **Property and Content Moderation:**

Admins monitor property listings to ensure data accuracy, authenticity, compliance with platform standards, and prevention of fraudulent activities. **Security and Data Privacy:**

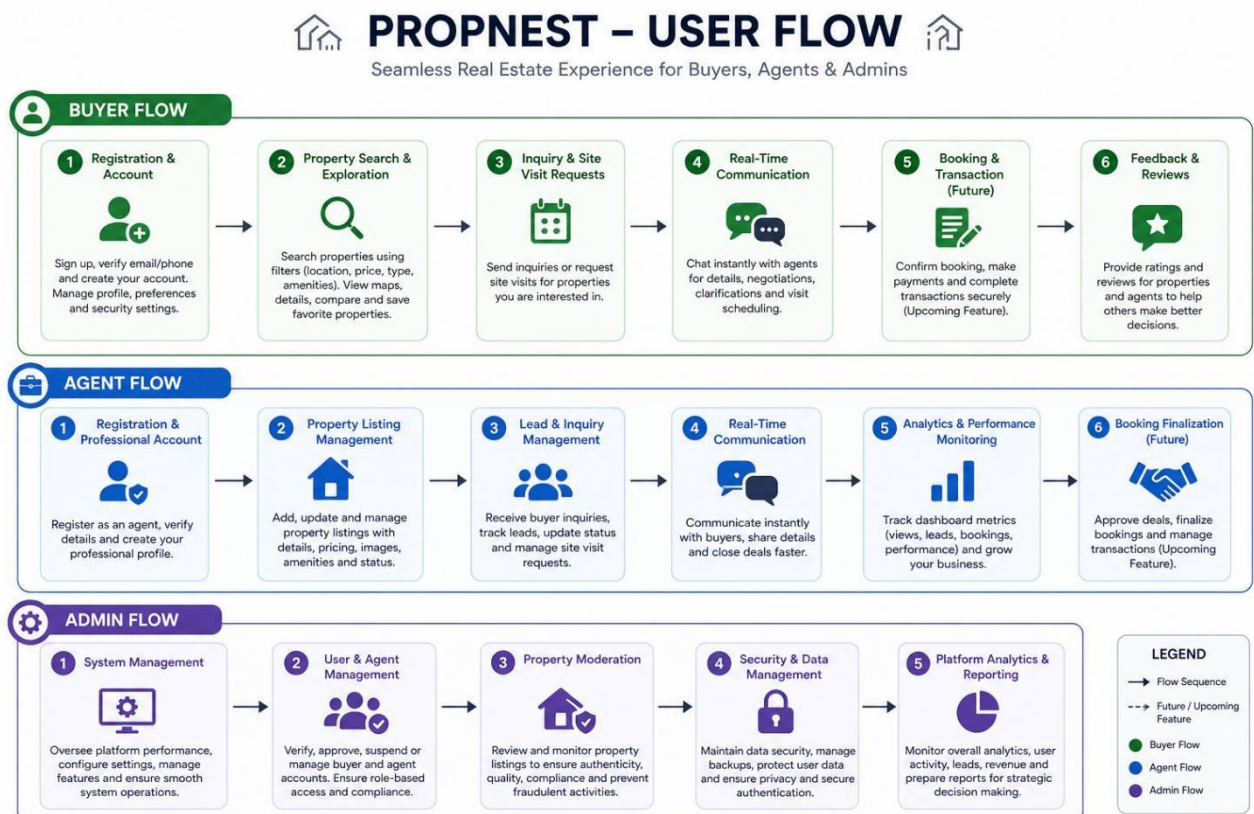
Admins are responsible for maintaining platform security through JWT authentication policies, database security, privacy protection, and secure management of user data.

### **Analytics and Operational Oversight:**

Admins monitor system-wide dashboards, user activity, lead generation metrics, and business performance to ensure efficient operation and strategic platform growth.

Overall, PropNest clearly defines the responsibilities of Buyers, Agents, and Admins to create a secure, transparent, and efficient real estate ecosystem that streamlines digital property management.

## **User Flow:**



## **Buyer Flow**

### **Registration & Account:**

Buyers create a secure PropNest account, register with personal details, select preferences, and manage profile information such as saved properties, contact details, and account security.

**Property Search & Exploration:**

Buyers search for properties using advanced filters such as location, budget, property type, amenities, and category. They can browse listings, view interactive maps, compare properties, and save favorites for future consideration.

**Inquiry & Site Visit Requests:**

Once a suitable property is found, buyers can submit inquiries, schedule site visits, or request additional information directly through the platform.

**Real-Time Communication:**

Buyers communicate instantly with agents through live chat to discuss property details, negotiate terms, clarify doubts, or coordinate visits.

**Booking & Transaction (Future Scope):**

Buyers can proceed with booking confirmations, payment processing, and transaction management as future platform enhancements are integrated.

**Feedback & Reviews:**

Buyers can provide reviews, ratings, and feedback on properties, agents, and their overall experience to support transparency and informed decision-making.

**Agent Flow****Registration & Professional Account:**

Agents register securely, create professional profiles, and manage agency details, certifications, and property portfolios.

**Property Listing Management:**

Agents add, update, manage, and monitor property listings with detailed information including pricing, location, amenities, images, and availability.

**Lead & Inquiry Management:**

Agents receive buyer inquiries, track lead progress, respond to visit requests, and manage communication efficiently through dedicated dashboards.

**Real-Time Communication:**

Agents engage directly with buyers via Socket.io-powered messaging for instant responses, better engagement, and faster conversions.

**Analytics & Performance Monitoring:**

Agents track dashboard metrics such as lead conversion, property performance, booking trends, and personalized business insights.

#### **Booking Finalization (Future Scope):**

Agents can eventually approve deals, finalize leases or sales, and manage booking workflows through enhanced platform modules.

### **Admin Flow**

#### **System Management:**

Admins oversee overall PropNest platform operations, maintain performance, configure settings, ensure scalability, and monitor security.

#### **User & Agent Management:**

Admins verify, approve, suspend, or manage buyer and agent accounts while maintaining role-based access and platform integrity.

#### **Property Moderation:**

Admins review and monitor property listings to ensure authenticity, quality, compliance, and fraud prevention.

#### **Security & Data Management:**

Admins maintain secure authentication systems, database integrity, privacy controls, and platform-wide data protection.

#### **Platform Analytics:**

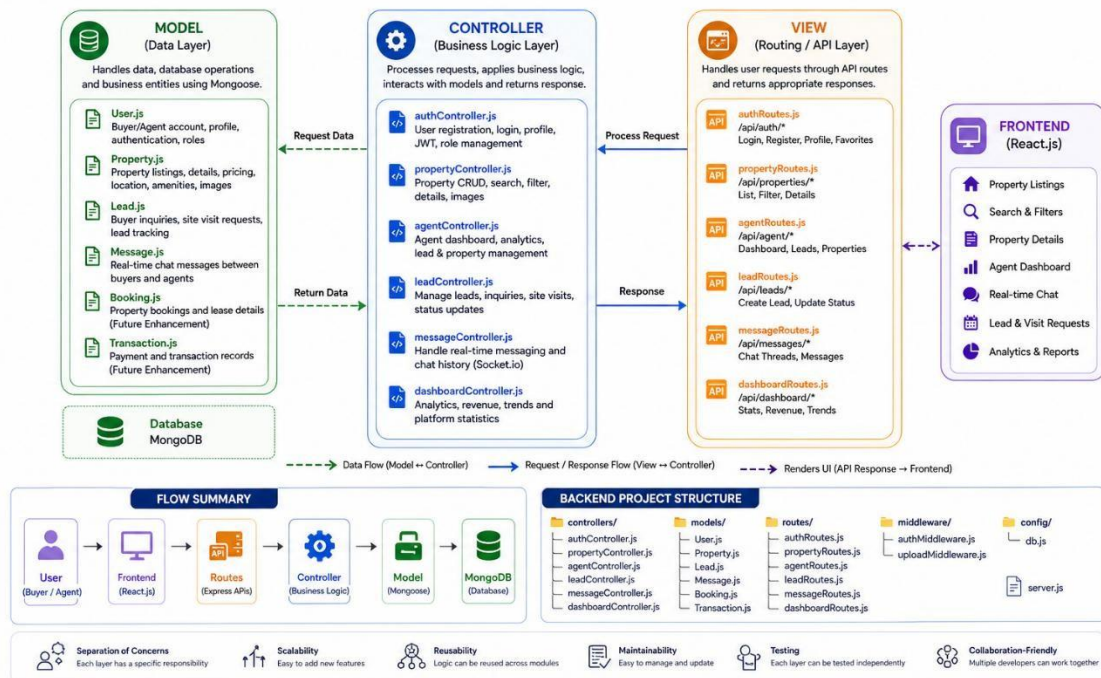
Admins analyze user engagement, operational performance, lead generation, and system-wide insights to improve platform growth and efficiency.

### **MVC PATTERN:**



## PROPNEST – MVC PATTERN

Model – View – Controller Architecture



The PropNest application follows the Model-View-Controller (MVC) architectural pattern, a highly efficient software design methodology that organizes the platform into three interconnected layers: Model, View, and Controller. This structured separation improves maintainability, scalability, security, and modular development for the full-stack real estate property management ecosystem.

### Model Layer (Data Layer)

The Model layer is responsible for managing all data-related logic, schema definitions, and database interactions. In PropNest, MongoDB serves as the primary database, while Mongoose is used to define schemas and manage structured data operations.

#### Key Models in PropNest:

- **User.js** → Handles buyer and agent account data, authentication, profiles, and role management.
- **Property.js** → Stores property listings including title, category, price, location, amenities, status, and images.
- **Lead.js** → Manages buyer inquiries, site visit requests, and lead tracking.
- **Message.js** → Stores real-time buyer-agent communication history.
- **Booking.js** → Defines booking and lease-related records (future enhancement).
- **Transaction.js** → Handles payment and financial transaction data (future enhancement).

These models ensure data consistency, validation, and secure interaction with the MongoDB database.

## Controller Layer (Business Logic Layer)

The Controller layer acts as the core processing engine of PropNest. It receives incoming client requests, validates data, applies business rules, interacts with models, and returns structured responses. **Key Controllers:**

- **authController.js** → Handles registration, login, JWT generation, profile updates, and role-based access.
- **propertyController.js** → Manages property CRUD operations, filtering, search, and property retrieval.
- **agentController.js** → Handles agent dashboard analytics, lead tracking, property management, and performance monitoring.
- **leadController.js** → Processes buyer inquiries, site visits, and lead status workflows.
- **messageController.js** → Handles messaging threads and Socket.io communication.
- **dashboardController.js** → Generates system-wide and personalized analytics data.

Controllers ensure separation between frontend requests and database operations, making business logic reusable and scalable.

## View Layer (Routing / API Layer)

In PropNest, the View layer is represented by API routes and frontend presentation.

### Backend Routing Layer:

Routes define RESTful API endpoints that manage HTTP requests (GET, POST, PUT, PATCH, DELETE) and connect them to appropriate controllers. **Key Routes:**

- **authRoutes.js** → Signup, login, profile, favorites
- **propertyRoutes.js** → Property listing, filtering, single property details
- **agentRoutes.js** → Agent dashboard, analytics, lead management, property CRUD
- **leadRoutes.js** → Inquiry and site visit requests
- **messageRoutes.js** → Chat threads and live communication
- **dashboardRoutes.js** → Revenue, stats, trends

### Frontend View:

React.js handles UI rendering through pages, dashboards, forms, maps, and messaging interfaces.

## Advantages of Using MVC in PropNest Separation of Concerns:

Each layer performs a dedicated role, improving readability, debugging, and maintenance. **Scalability:**

New modules such as payment gateways, booking systems, or tenant management can be added easily. **Reusability:**

Controllers and models can serve multiple routes and features without code duplication. **Testing:**

Each component can be tested independently, enabling better quality assurance.

**Collaboration-Friendly:**

Frontend and backend teams can work simultaneously on separate layers.

## **Project Setup and Configuration**

### **Creating Project Folder**

1. Create a new folder named **PropNest**

2. Inside the folder, create two subfolders:

**Frontend**

**Backend**

3. Open the project in Visual Studio Code

### **Frontend Setup (React + Vite) Open**

**Frontend folder:**

npm create vite@latest . -- --template react **Select:**

- Framework: React
- Variant: JavaScript

**Install dependencies:**

npm install

**Start**

**frontend:** npm

run dev

### **Backend Setup (Node + Express)**

**Open Backend folder:** npm

init -y

**Install dependencies:**

npm install express mongoose dotenv cors bcryptjs jsonwebtoken socket.io

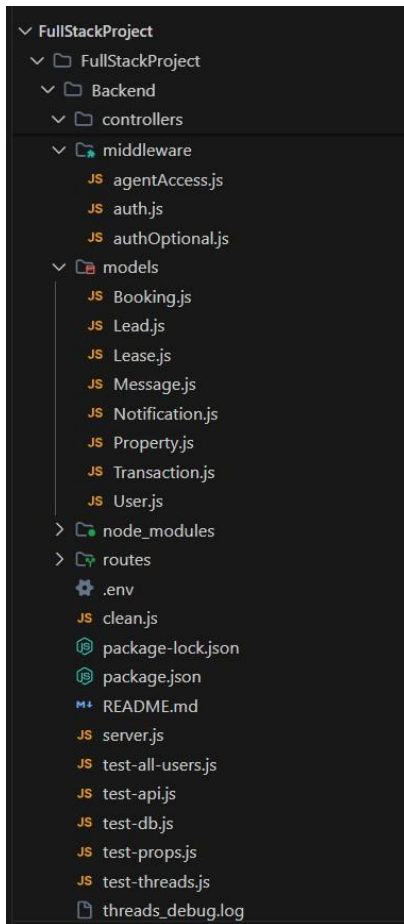
**Create:**

- server.js
- **Create folders:**
- Models
- Controllers
- Routes
- Middleware
- config

## **BACKEND DEVELOPMENT STRUCTURE:**



## Backend Reference Video:



### Controllers:

- **authController.js** → User authentication and profile logic
- **propertyController.js** → Property search, filters, CRUD
- **agentController.js** → Agent dashboard and management
- **leadController.js** → Inquiry and visit logic
- **messageController.js** → Real-time messaging

### Config:

- **db.js** → MongoDB connection setup

### Middleware:

- **authMiddleware.js** → JWT authentication and role validation
- **uploadMiddleware.js** → Handles property image uploads

### Models:

#### User:

- **Users.js Property:**

- Property.js **Lead:**
- Lead.js **Message:**
- Message.js **Booking:**
- Booking.js **Transaction:**
- Transaction.js

#### **Routes:**

- authRoutes.js
- propertyRoutes.js
- agentRoutes.js
- leadRoutes.js
- messageRoutes.js
- dashboardRoutes.js

## **DATABASE DEVELOPMENT:**

### **Configure MongoDB:**

PropNest uses MongoDB as its primary NoSQL database to manage users, properties, leads, messaging, bookings, and transaction data. MongoDB is integrated using Mongoose, which provides schema-based modeling and structured database interactions for the full-stack real estate platform.

### **Install Mongoose:**

In the Backend project directory, install Mongoose using: npm  
install mongoose

### **Create Database Connection:**

To ensure PropNest performs backend operations securely, MongoDB must be connected before handling API requests.

### **Example Database Connection (config/db.js):**

```
const mongoose = require("mongoose");

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log("MongoDB Connected Successfully");
  } catch (error) {
    console.error("Database Connection Failed:", error.message);
    process.exit(1);
  }
};
```

module.exports = connectDB; **Server**

### **Integration:**

In server.js, connect the database before starting the server:

```
const connectDB = require("../config/db"); connectDB();
```

### **Create Schemas & Models:**

Using the ER Diagram structure, PropNest defines multiple schemas to organize buyer, agent, property, lead, and communication data efficiently.

#### **1) Users.js → Schema/model for Buyers and Agents**

Stores account details, authentication, and role-based information.

```

JS User.js x
FullStackProject > Backend > models > JS User.js > ...
1  const mongoose = require('mongoose');
2
3  const userSchema = new mongoose.Schema({
4    email: {
5      type: String,
6      required: true,
7      unique: true,
8      trim: true,
9      lowercase: true,
10   },
11   password: {
12     type: String,
13     required: true,
14   },
15   full_name: {
16     type: String,
17     required: true,
18   },
19   /** Primary access level: admin, agent, or normal user (buyer). */
20   role: {
21     type: String,
22     enum: ['admin', 'agent', 'user'],

```

### Key Fields:

- Name
- Email
- Password
- Role (Buyer/Agent/Admin)
- Phone
- Profile Image
- Saved Properties

**2) Property.js → Schema/model for Property Listings** Stores complete property details managed by agents.

```

JS Property.js X
FullStackProject > Backend > models > JS Property.js > ...
1  const mongoose = require('mongoose');
2
3  const propertySchema = new mongoose.Schema({
4    title: { type: String, required: true },
5    location: { type: String, required: true },
6    address: (property) type: StringConstructor
7    city: { type: String, required: true },
8    state: { type: String, default: 'Maharashtra' },
9    price: { type: Number, required: true },
10   priceUnit: { type: String, required: true },
11   type: { type: String }, // Keep for compatibility
12   listing_type: { type: String, default: 'sale' },
13   category: { type: String, required: true },
14   bedrooms: { type: Number, default: 0 },
15   bathrooms: { type: Number, default: 0 },
16   area: { type: Number }, // Keep for compatibility
17   area_sqft: { type: Number, required: true },
18   areaUnit: { type: String, default: 'sqft' },
19   image: { type: String, required: true },
20   images: [{ type: String }],
21   amenities: [{ type: String }],
22   status: { type: String, default: 'available' },
23   featured: { type: Boolean, default: false },
24   trending: { type: Boolean, default: false },
25   is_verified: { type: Boolean, default: false },
26   is_new: { type: Boolean, default: false },

```

### Key Fields:

- Property\_ID
- Agent\_ID
- Title
- Category
- Type
- Location
- Price
- Amenities
- Images
- Status

**3) Lead.js → Schema/model for Buyer Inquiries & Site Visits Tracks**  
 buyer interest and inquiry workflow.

```

JS Lead.js x
FullStackProject > Backend > models > JS Lead.js > [e] leadSchema > [e] email
1  const mongoose = require('mongoose');
2
3  const leadSchema = new mongoose.Schema({
4    property_id: {
5      type: mongoose.Schema.Types.ObjectId,
6      ref: 'Property',
7      required: true
8    },
9    agent_id: {
10     type: mongoose.Schema.Types.ObjectId,
11     ref: 'User',
12     required: true
13   },
14   // Optional: if lead was submitted by a logged-in buyer user
15   buyer_user_id: {
16     type: mongoose.Schema.Types.ObjectId,
17     ref: 'User',
18     default: null,
19   },
20   name: {
21     type: String,
22     required: true
23   },
24   email: [
25     type: String,

```

#### Key Fields:

- Lead\_ID
- Buyer\_ID
- Property\_ID
- Agent\_ID
- Inquiry Type
- Visit Date
- Status

**4) Message.js → Schema/model for Real-Time Chat Stores**  
 buyer-agent communication threads.

```

JS Message.js X
FullStackProject > Backend > models > JS Message.js > ...
1  const mongoose = require('mongoose');
2
3  const messageSchema = new mongoose.Schema({
4    lead_id: {
5      type: mongoose.Schema.Types.ObjectId,
6      ref: 'Lead',
7      required: true,
8    },
9    sender_id: {
10     type: mongoose.Schema.Types.ObjectId,
11     ref: 'User',
12     required: true,
13   },
14   sender_role: {
15     type: String,
16     enum: ['buyer', 'agent'],
17     required: true,
18   },
19   text: {
20     type: String,
21     required: true,

```

#### Key Fields:

- Message\_ID
- Lead\_ID
- Sender\_ID
- Receiver\_ID
- Message Content
- Timestamp

**5) Booking.js → Schema/model for Property Booking (Future Enhancement) Handles booking and lease workflows.**

```

JS Booking.js X
FullStackProject > Backend > models > JS Booking.js > bookingSchema > ⚙️ userId
1  const mongoose = require('mongoose');
2
3  const bookingSchema = new mongoose.Schema({
4    propertyId: {
5      type: mongoose.Schema.Types.ObjectId,
6      ref: 'Property',
7      required: true
8    },
9    userId: {
10     type: mongoose.Schema.Types.ObjectId,
11     ref: 'User',
12     required: true
13   },
14   agentId: {
15     type: mongoose.Schema.Types.ObjectId,
16     ref: 'User',
17     required: true
18   },
19   bookingDate: {
20     type: Date,
21     default: Date.now
22   },
23   visitDate: {
24     type: Date,
25     required: true

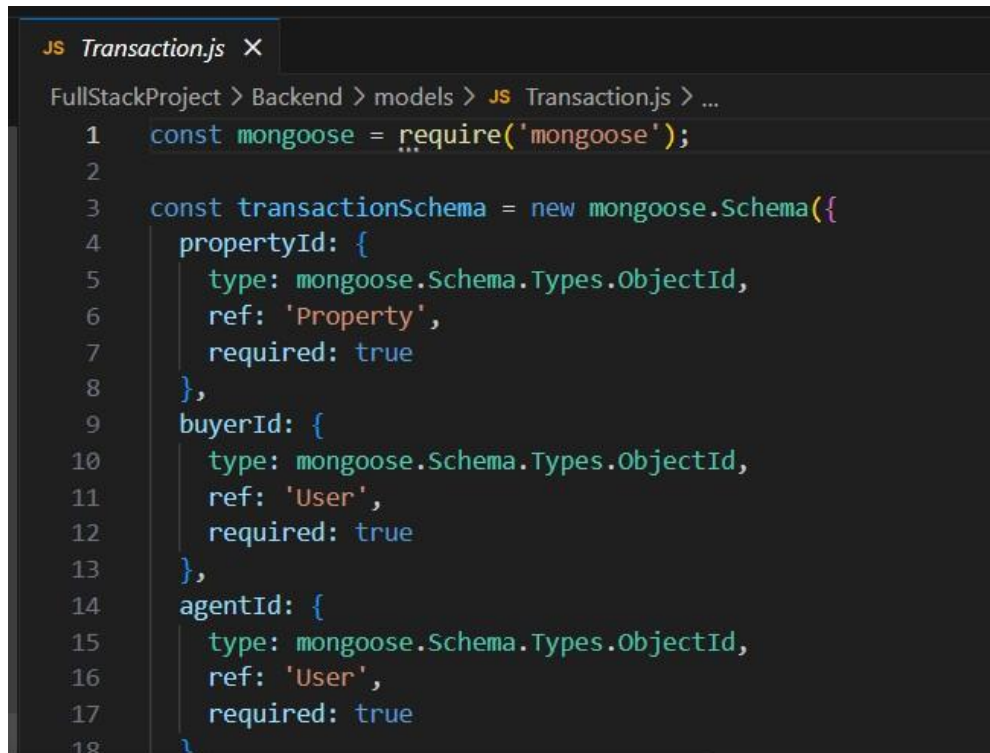
```

**Key Fields:**

- Booking\_ID
- Buyer\_ID
- Property\_ID
- Booking Date
- Contract Status

**6) Transaction.js → Schema/model for Payment Records (Future Enhancement)**

Tracks payment workflows.



```
JS Transaction.js X
FullStackProject > Backend > models > JS Transaction.js > ...
1  const mongoose = require('mongoose');
2
3  const transactionSchema = new mongoose.Schema({
4    propertyId: {
5      type: mongoose.Schema.Types.ObjectId,
6      ref: 'Property',
7      required: true
8    },
9    buyerId: {
10     type: mongoose.Schema.Types.ObjectId,
11     ref: 'User',
12     required: true
13   },
14   agentId: {
15     type: mongoose.Schema.Types.ObjectId,
16     ref: 'User',
17     required: true
18   }
19 }
```

**Key Fields:**

- Transaction\_ID
- Booking\_ID
- Amount
- Payment Method
- Payment Status



## FRONT-END DEVELOPMENT:

The PropNest frontend is developed using React.js with Vite, Tailwind CSS, Shadcn UI, React Router, and Context API to create responsive, modular, and interactive user experiences.

### Frontend Structure:

#### Buyer Section (src/buyer/)

- **BuyerHome.jsx**

Buyer dashboard with property recommendations, recent activity, and quick navigation.

- **BuyerLogin.jsx**

Buyer authentication page.

- **BuyerSignup.jsx**

Buyer registration page.

- **BuyerNavbar.jsx**

Navigation for buyer dashboard.

- **Properties.jsx**

Displays all available properties with filters and search.

- **PropertyDetail.jsx**

Detailed view of a selected property.

- **SavedProperties.jsx**

Displays favorite/saved listings.

- **CompareProperties.jsx**

Side-by-side comparison of selected properties.

- **BuyerMessages.jsx**

Real-time chat interface for buyer-agent communication.

#### Agent Section (src/agent/)

- **AgentDashboard.jsx**

Main dashboard with analytics and metrics.

• **AgentLogin.jsx** Agent login page.

- **AgentSignup.jsx**

Agent registration page.

- **AgentNavbar.jsx**

Navigation bar for agent dashboard.

- **PostProperty.jsx**

UI for adding new property listings.

- **PropertyManager.jsx**

Manage existing properties.

- **LeadTracker.jsx**

Track buyer inquiries and site visits.

- **AgentMessages.jsx**

Real-time buyer communication.

- **AgentAnalyticsDashboard.jsx**

Performance, revenue, and lead conversion insights.

### **Admin Section (src/admin/)**

- **AdminDashboard.jsx**

Platform management dashboard. •

**UserManagement.jsx**

Manage buyers and agents.

- **PropertyModeration.jsx** Review and moderate listings. • **SystemAnalytics.jsx**  
Monitor platform performance.

### **Shared Components (src/components/)**

- **Home.jsx**

Landing page

- **Navbar.jsx**

Global navigation

- **Footer.jsx**

Footer component

- **MapView.jsx**

Leaflet property mapping

- **SearchFilters.jsx**

Advanced filtering UI

- **ChatWindow.jsx**

Reusable messaging component

- **ui/**

Reusable buttons, cards, dialogs, forms, and modal components.

### **Advantages of This Development Structure:**

#### **Modular Development:**

Separate Buyer, Agent, Admin, and Shared components.

#### **Scalability:**

Easy integration of future booking, payment, and tenant systems.

#### **Reusability:**

Reusable components reduce redundancy.

### Security:

Role-based frontend and backend architecture.

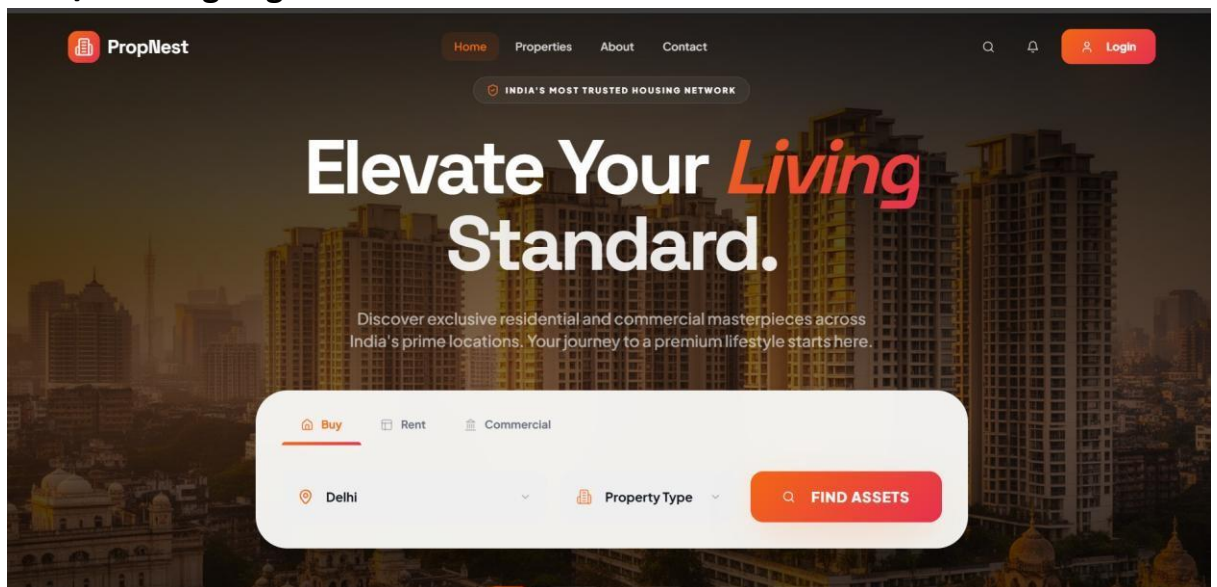
### Maintainability:

Clean code organization for long-term project growth.

Overall, PropNest's database and frontend development architecture create a highly scalable, enterprise-ready real estate ecosystem that efficiently supports buyers, agents, and administrators through structured data management and modern user interfaces.

## Output ScreenShots:

### 1. Home / Landing Page



### 2. User Registration & Login Page



# PropNest

Join thousands of happy homeowners and agents across India.

## Create Account

Start your property journey today

Home Buyer

Agent

Full Name

John Doe

Email

you@example.com

Confirm Email

Re-enter your email

Phone

+91 9876543210

Password

Min 6 characters

Confirm Password

Re-enter password

Create Account



# PropNest

Join thousands of happy homeowners and agents across India.

## Create Account

Start your property journey today

Home Buyer

Agent

Full Name

John Doe

Email

you@example.com

Confirm Email

Re-enter your email

Phone

+91 9876543210

Password

Min 6 characters

Confirm Password

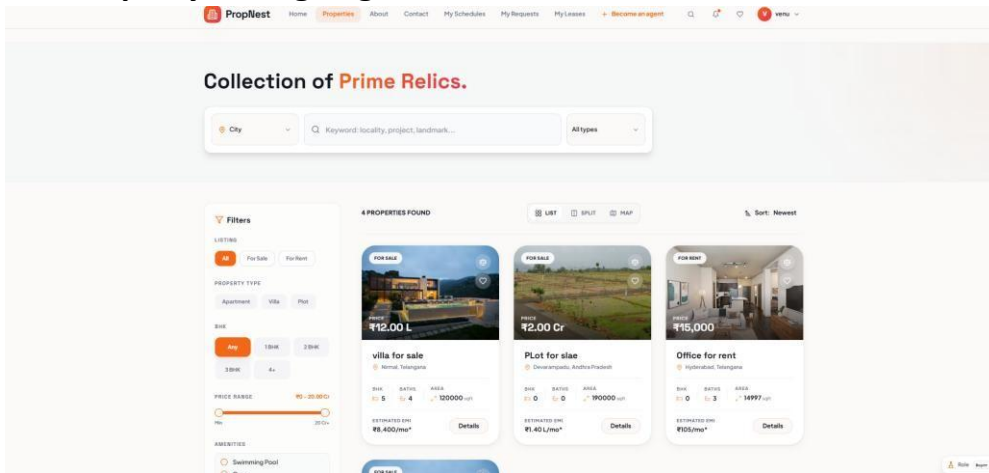
Re-enter password

Register as Agent

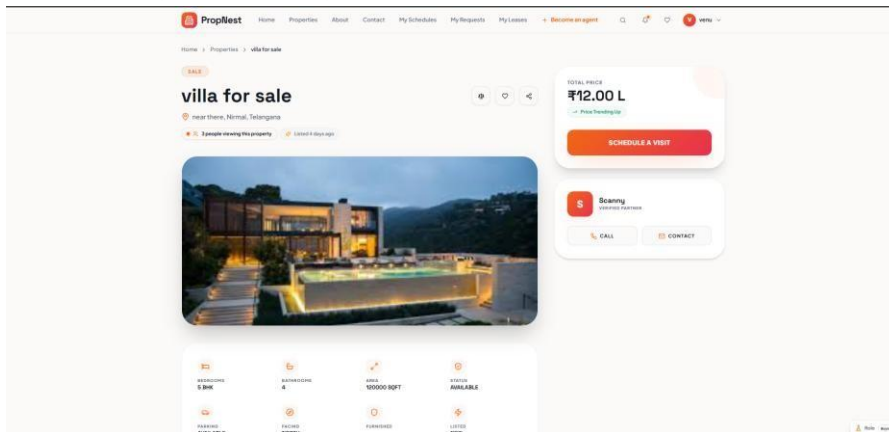
### 3. Buyer Dashboard



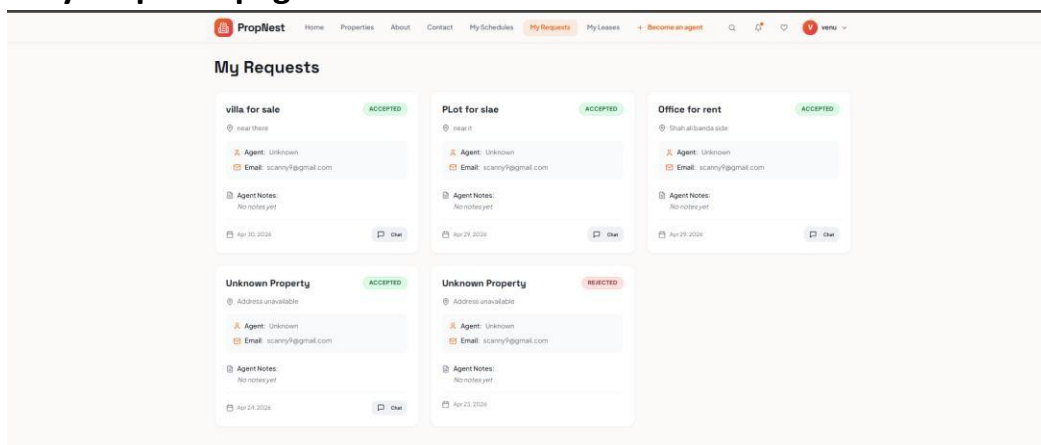
## 4. Property Listing Page



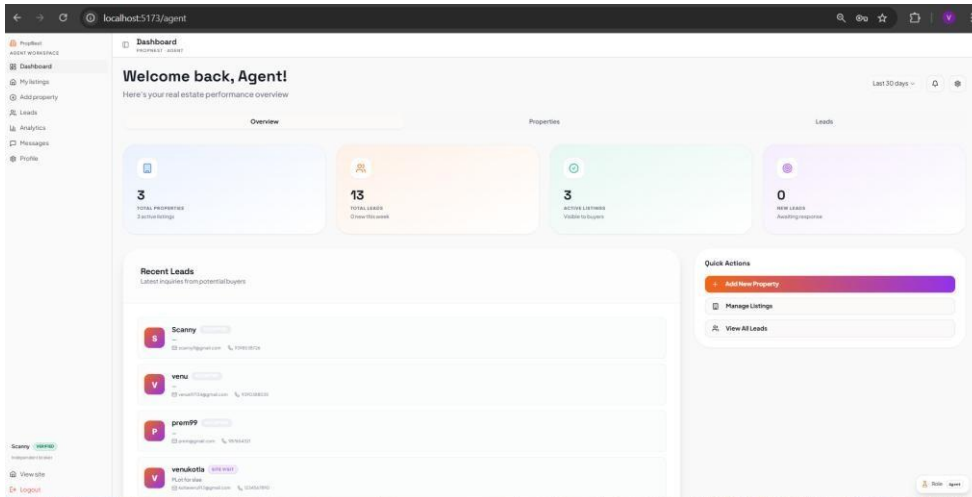
## 4. Property Detail Page



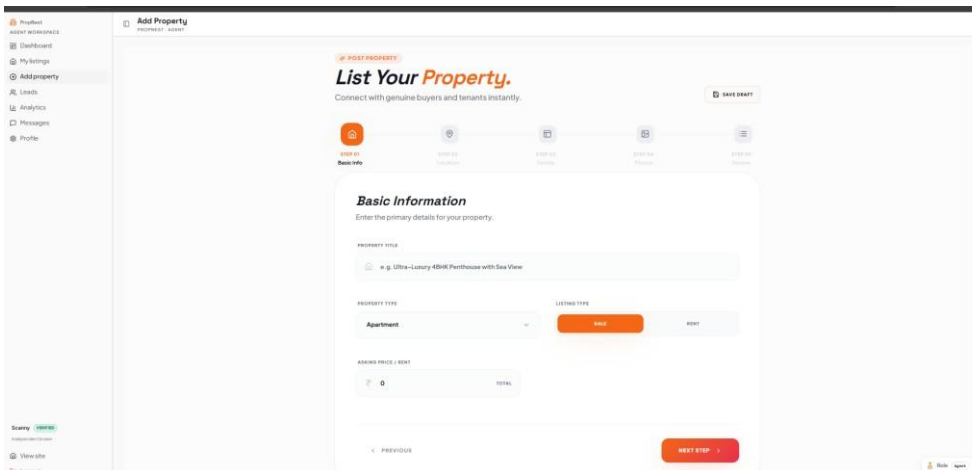
## 5. My Requests page



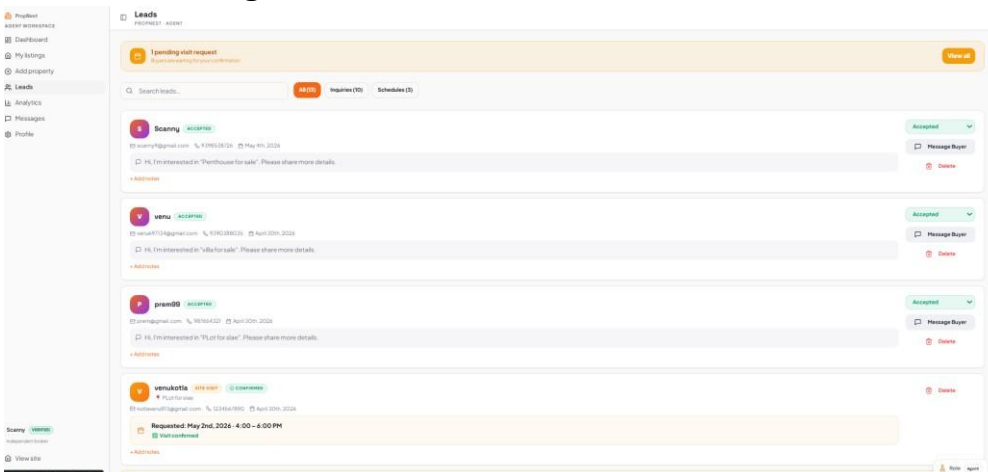
## 6. Agent Dashboard



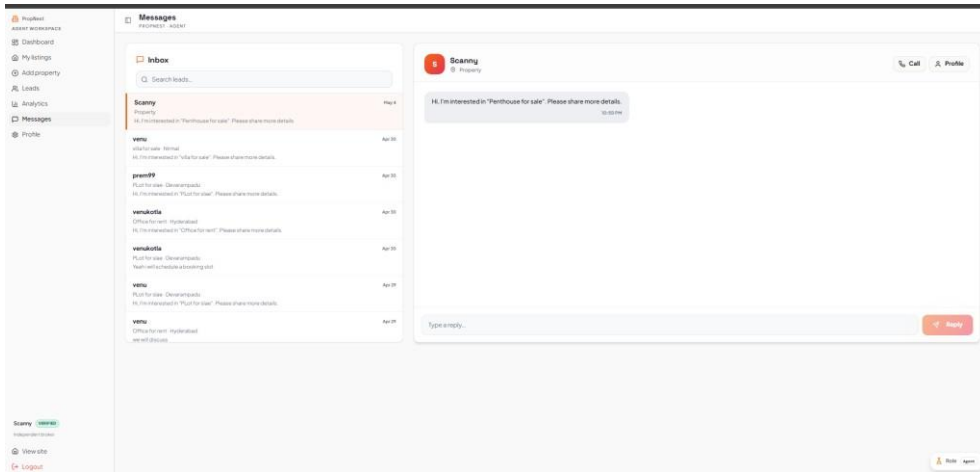
## 7. Add / Manage Property Page



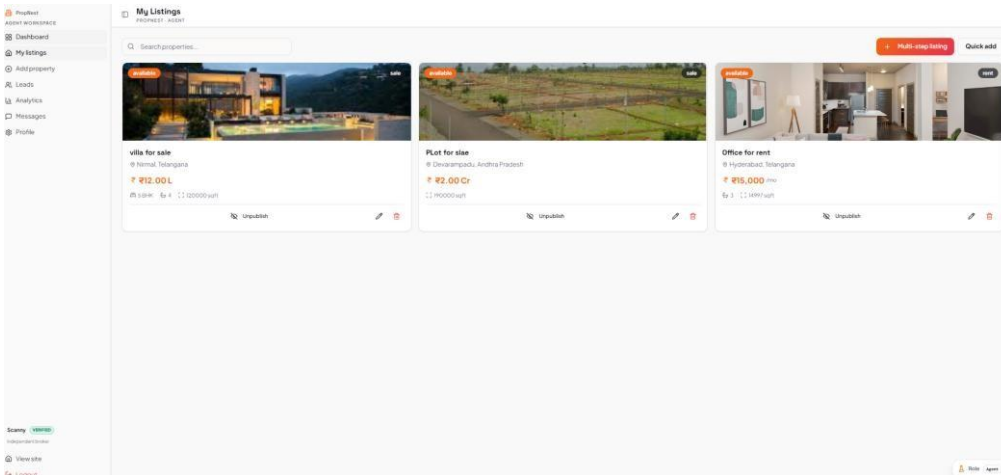
## 8. Lead Tracking Screen



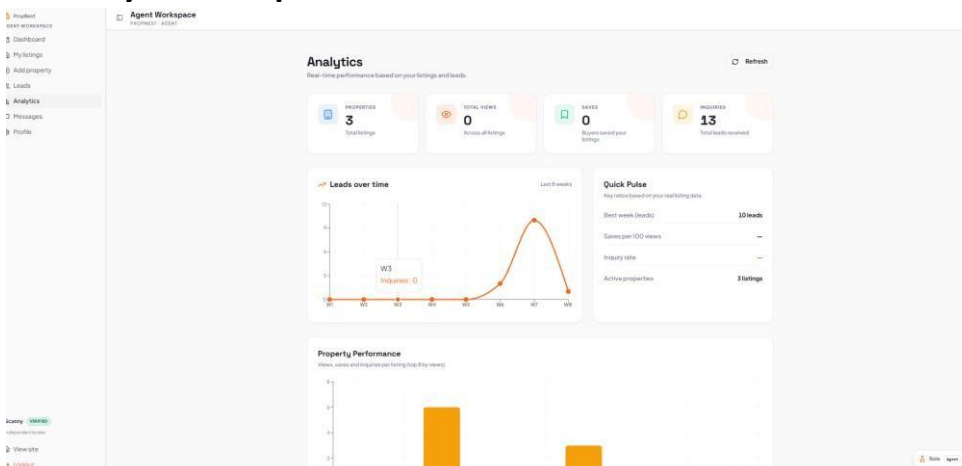
## 9. Real-Time Messaging Interface



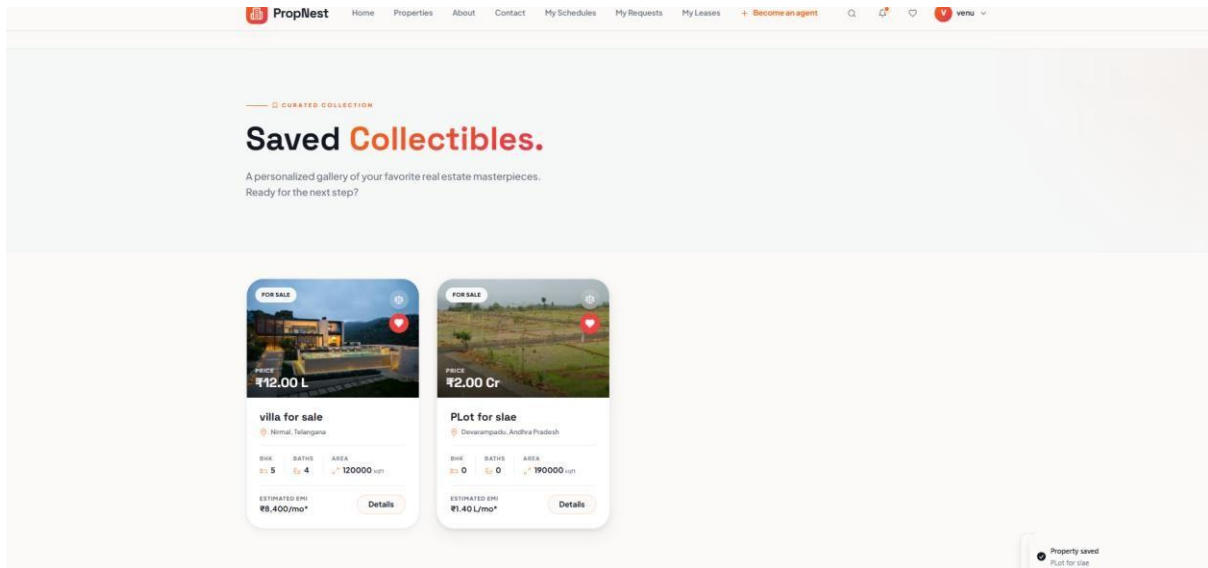
## 10. Saved Properties Page



## 11. Analytics & Reports Dashboard



## 13. Wishlist Page



## Demo Video Link:

<https://drive.google.com/file/d/12whnThd8kpROxem5Gfnz8Dliptnoqin/view?usp=sharing>

## Code Drive Link:

[https://drive.google.com/file/d/1o\\_V5knakl1wB\\_JDTWJediQBHoTfarCbe/view?usp=sharing](https://drive.google.com/file/d/1o_V5knakl1wB_JDTWJediQBHoTfarCbe/view?usp=sharing)